



**ΠΑΝΕΠΙΣΤΗΜΙΟ
ΠΑΤΡΩΝ**
UNIVERSITY OF PATRAS

**ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ
ΤΕΧΝΟΛΟΓΙΑΣ ΥΠΟΛΟΓΙΣΤΩΝ**

ΟΝΟΜΑΤΕΠΩΝΥΜΟ ΦΟΙΤΗΤΗ: ΣΑΛΙΑΡΗΣ ΓΕΩΡΓΙΟΣ

ΑΜ: 1092584

ΑΚΑΔΗΜΑΪΚΟ ΕΤΟΣ: 2024-2025

ΜΑΘΗΜΑ: ΕΙΣΑΓΩΓΗ ΣΤΗ ΡΟΜΠΟΤΙΚΗ

**ΥΠΕΥΘΥΝΟΣ ΚΑΘΗΓΗΤΗΣ: ΚΩΝΣΤΑΝΤΙΝΟΣ
ΧΑΤΖΗΛΥΓΕΡΟΥΔΗΣ**

ΑΤΟΜΙΚΗ ΕΡΓΑΣΙΑ #3

A. Σκοπός / Περιγραφή της εργασίας

Η παρούσα εργασία συνιστά συνέχεια των 2^η προηγούμενων εργασιών, υπό την έννοια ότι έχουμε το ίδιο *manipulation scenario* (ένα κουτί με μία οπή στο κέντρο του, έναν κύλινδρο, και ένα ρομποτικό βραχίονα *Franka Panda*, στόχος του οποίου είναι πιάσει τον κύλινδρο και με κατάλληλες κινήσεις να τον αφήσει πάνω από την οπή, με αποτέλεσμα ο κύλινδρος να περάσει μέσα από την οπή. Ειδικότερα, αφού καθοριστούν οι εμπλεκόμενοι πίνακες μετασχηματισμού, δημιουργούνται 2 τροχιές για τον *end-effector*: 1 για να φτάσει τον κύλινδρο από το αρχικό του *configuration* και να τον κάνει *grasp*, και έπειτα αφού τον έχει πιάσει, να τον αφήσει πάνω από την τρύπα του κουτιού. Γι' αυτό το λόγο, αξιοποιείται ένας *PI* ελεγκτής στόχος του οποίου είναι να ελαχιστοποιήσει το σφάλμα ανάμεσα στα πραγματικά και στα επιθυμητά *poses* του *end-effector*. Τέλος, αποτυπώνουμε τα παραπάνω σε έναν κατάλληλο *kinematic simulator* μέσω κώδικα. Να σημειωθεί ότι τα ερωτήματα τα οποία έχουν απαντηθεί στις 2 προηγούμενες εργασίες αναφέρονται επιγραμματικά ή/και δεν εξηγούνται με αναλυτικό τρόπο.

B. Ζητούμενα της άσκησης

Ερώτημα 1: Ακριβώς ίδιο με τις προηγούμενες εργασίες

$$\text{Ερώτημα 2: Ομοίως } T_1 = I_4 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1)$$

$$\text{Ερώτημα 3: } T_{\text{world,cyl}} = \begin{bmatrix} 1 & 0 & 0 & 69.50 \\ 0 & 1 & 0 & 9.50 \\ 0 & 0 & 1 & 2.50 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2)$$

$$\text{Ερώτημα 4: } T_{\text{world,cylnew}} = \begin{bmatrix} 0 & 0 & -1 & 50 \\ 0 & 1 & 0 & 50 \\ 1 & 0 & 0 & 45 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3)$$

Ερώτημα 5: Σχεδιασμός τροχιών για τον *end-effector*.

Εδώ καλούμαστε να κατασκευάσουμε 2 *trajectories* για το σύστημα, προκειμένου το τελευταίο να φέρει εις πέρας τα 2 manipulation tasks. Να πιάσει τον κύλινδρο (*trajectory #1*) και ύστερα να τον αφήσει να πέσει πάνω από την τρύπα του κουτιού-αφού ο κύλινδρος έχει λάβει τον κατάλληλο προσανατολισμό (*trajectory #2*). Επομένως, για κάνουμε generate τα παραπάνω, κάνουμε χρήση των *cubic splines* (γενικά αποφεύγουμε τα πολυώνυμα 4ου βαθμού καθώς μας οδηγούν σε *overparameterized* συστήματα, πράγμα το οποίο σημαίνει ότι έχουμε πολλαπλές λύσεις, άρα και περισσότερες αποφάσεις τις οποίες πρέπει να λάβουμε). Για το 1ο trajectory θεωρούμε ότι (οι «αρχικές συνθήκες»):

$$T_{s_1} = T_{world, cyl} = \begin{bmatrix} \sqrt{2}/2 & \sqrt{2}/2 & 0 & 0 \\ -\sqrt{2}/2 & \sqrt{2}/2 & 0 & 10.3 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} (4)$$

$$V_s = [0] \text{ (6x1 διάνυσμα)} (5)$$

Τελικές συνθήκες:

V_g αυθαίρετο και για τον πίνακα προορισμού θα έχουμε ότι:

$$T_{g_1} = T_{world, cyl} = \begin{bmatrix} 1 & 0 & 0 & 69.50 \\ 0 & 1 & 0 & 9.50 \\ 0 & 0 & 1 & 2.50 \\ 0 & 0 & 0 & 1 \end{bmatrix} (5)$$

αφού θέλουμε ο *end-effector* να πιάσει τον κύλινδρο. Τώρα, προκειμένου να σχεδιάσουμε την 1^η τροχιά η οποία επιτρέπει στον *end-effector* να πιάσει τον κύλινδρο και μόνο αυτό, ακολουθούμε την παρακάτω διαδικασία: (χρησιμοποιούμε *cubic spline*-«πολυώνυμο» 3^{ου} βαθμού δηλαδή).

$$T_{d_1}(t) = \exp(c_{3_{hat}}t^3 + c_{2_{hat}}t^2 + c_{1_{hat}}t + c_{0_{hat}}) (6)$$

Όπου στην (6) T_d είναι η επιθυμητή τροχιά. Επίσης:

$$T'_{d_1}(t) = (3c_{3_{hat}}t^2 + c_{2_{hat}}t + c_{0_{hat}}) * T_d(t) (7) \text{ λόγω ιδιοτήτων παραγώγισης}$$

$$T^2_{d_1}(t) = (6c_{3_{hat}}t + 2c_{2_{hat}}) * T'_d(t) (8) \text{ (ο δείκτης (i) συμβολίζει την i - οστή παράγωγο ως προς χρόνο)}$$

$$T^3_{d_1}(t) = (6c_{3_{hat}}) * T^2_d(t) (9)$$

Ενώ για τις αντίστοιχες αρχικές συνθήκες θα είναι:

$$T_{d_1}(0) = \exp(c_{0_{hat}}) (11)$$

$$T'_{d_1}(0) = \exp(c_{1_{hat}}) (12)$$

$$T^2_{d_1}(0) = \exp(c_{2_{hat}}) (13)$$

$$T^3_{d_1}(0) = \exp(c_{3_{hat}}) (14)$$

Στο σημείο αυτό αξίζει να σταθούμε για λίγο στις διαστάσεις των c_{ihat} .

Από την σχέση (11), άμεσα καταλαβαίνουμε ότι τα c_{ihat} είναι πίνακες διαστάσεων 4×4 (για να μπορούν να βγαίνουν οι σχετικές πράξεις). Σύμφωνα με την θεωρία θα είναι:

$$c_{ihat} = \begin{bmatrix} a_{hat} & b \\ \mathbf{0} & 0 \end{bmatrix} \quad (15)$$

Όπου στη (15) το a_{hat} είναι ο *skew-symmetric matrix* του διανύσματος $a = [a_1 \ a_2 \ a_3]^T$ (16). Είναι επομένως:

$$a_{hat} = \begin{bmatrix} 0 & -\alpha_3 & \alpha_2 \\ \alpha_3 & 0 & -\alpha_1 \\ -\alpha_2 & \alpha_1 & 0 \end{bmatrix} \quad (16)$$

Το δε διάνυσμα b έχει διαστάσεις 3×1 , ενώ το $\mathbf{0}$ στην (15) είναι μια *ημι-γραμμή* με 3 μηδενικά (το δε δίπλα μηδενικό στην (15) αντιστοιχεί στο ψηφίο 0).

Άρα, θα έχουμε ότι:

$$c_0 = \log^{invhat}(T_s) = \tau_s \quad (17)$$

Στην (17) όμως εφαρμόζουμε τον *inverse skew symmetric matrix operator* για έναν πίνακα 4×4 . Αν λοιπόν έχουμε έναν πίνακα A διαστάσεων 4×4 τέτοιον ώστε:

$$A = \begin{bmatrix} 0 & a & b & c \\ -a & 0 & d & e \\ -b & -d & 0 & f \\ -c & -e & -f & 0 \end{bmatrix} \quad (18), \text{ τότε:}$$

$$\log^{invhat}(A) = [a \ b \ c \ d \ e \ f]^T \quad (19)$$

Για τα υπόλοιπα δε διανύσματα c_1, c_2, c_3 καταλήγουμε στα εξής:

$$c_1 = V_s \quad (20)$$

$$c_2 = \frac{3\tau_g}{T^2} - \frac{3\tau_s}{T^2} - \frac{2V_s}{T} - \frac{V_g}{T} \quad (21)$$

$$c_3 = -\frac{2\tau_g}{T^3} + \frac{2\tau_s}{T^3} + \frac{V_s}{T^2} + \frac{V_g}{T^2} \quad (22)$$

$$\log^{\text{invhat}}(T_g) = \tau_g \quad (23)$$

όπου στην (21) V_s και V_g αποτελούν τα *twists* αφετηρίας και προορισμού (θεωρούμε ότι το $V_s = [0]$ (24), για το δε V_g θεωρούμε μια αυθαίρετη τιμή-ούτε πολύ μικρή αλλά και ούτε και πολύ μεγάλη για ευνόητους λόγους) ενώ προφανώς ισχύει ότι

$$V_i = [\omega_i \ v_i]^T \varepsilon R^{6 \times 1} \quad (25)$$

Ενώ T είναι ο χρόνος στον οποίο θεωρούμε εμείς ότι ο *gripper* οριακά πιάνει τον κύλινδρο (αυθαίρετος και αυτός).

Για το 2^ο επιθυμητό *trajectory* (ο *end-effector* έχει πιάσει τον κύλινδρο και θέλει να τον μετακινήσει με τέτοιο τρόπο ώστε ο τελευταίος να εισέρχεται οριακά στην οπή του κόκκινου κουτιού), ακολουθούμε ακριβώς την ίδια διαδικασία, μόνο που αλλάζουν οι πίνακες T_s και T_g (πλέον ο *end-effector* και ο κύλινδρος ακολουθούν κοινή τροχιά). Ειδικότερα:

$$T_{s_2} = T_{g_1} = \begin{bmatrix} 1 & 0 & 0 & 69.50 \\ 0 & 1 & 0 & 9.50 \\ 0 & 0 & 1 & 2.50 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (26)$$

$$T_{g_2} = T_{\text{world}, \text{cylnew}} = \begin{bmatrix} 0 & 0 & -1 & 50 \\ 0 & 1 & 0 & 50 \\ 1 & 0 & 0 & 45 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (27)$$

Οι τιμές των εκάστοτε παραμέτρων (διανυσμάτων) υπολογίζονται από τον σχετικό κώδικα.

Ερώτημα 6: Δημιουργία *Kinematic Simulator* (Ο σχετικός κώδικας παρατίθεται στο ερώτημα 8).

Αρχικά ξεκινάμε από τα πιο βασικά. Κάθε χρονική στιγμή (καλύτερα: σε κάθε επανάληψη του σχετικού *loop*), τα *joint positions* (γωνίες των αρθρώσεων) θα δίνονται από την σχέση (*Euler Integration*):

$$q_{k+1} = q_k + q' * \Delta t (28)$$

Όμως γνωρίζουμε ότι:

$$q' = v = J_b^\dagger * V_b (29)$$

με τον όρο J_b να παραπέμπει στην Ιακωβιανή ως προς το *body frame* του *end-effector* και $J_b^\dagger$ είναι ο Moore-Penrose ψευδοαντίστροφος πίνακας της J_b

$$J_b^\dagger = J_b^T (J_b J_b^T)^{-1} (17), n = 7 > m = 6 (30)$$

Για τον υπολογισμό του $[V_b] = \begin{bmatrix} \widehat{\omega}_b & v_b \\ \mathbf{0} & 0 \end{bmatrix}$ (είναι $V_b = [\omega_b \ v_b]^T$ (31) θα ισχύει η γνωστή σχέση:

$$T_{wb}^{-1} * T'_{wb} = [V_b] = \begin{bmatrix} \widehat{\omega}_b & v_b \\ \mathbf{0} & 0 \end{bmatrix} (32)$$

όπου στην παραπάνω εξίσωση ο πίνακας T_{wb} προκύπτει από τις σχετικές εξισώσεις των *Forward Kinematics* (έχουν παρουσιαστεί στη 2^η εργασία γι' αυτό το λόγο δεν αναφέρονται και στην παρούσα) για τον *end-effector* (σημείωση: ο T_{wb} δεν αντιστοιχεί στον πίνακα T_d).

Ερώτημα 7: Δημιουργία κατάλληλου *velocity controller* ώστε να ακολουθεί τις 2 τροχιές που ορίστηκαν.

Είναι προφανές ότι το σύστημα δε δύναται να ακολουθήσει επακριβώς την επιθυμητή τροχιά. Γι' αυτό λόγο προβαίνουμε στην κατασκευή ενός ελεγκτή ο οποίος θα συνδράμει στην επίτευξη της επιθυμητής τροχιάς σε σχετικά ικανοποιητικό βαθμό.

Στην προκειμένη περίπτωση χρησιμοποιούμε ελεγκτή τύπου *PI*. Η εξίσωση που περιγράφει τον ελεγκτή είναι η ακόλουθη:

$$V_{PI}(t) = K_p * X_e(t) + K_i * \int_0^t X_e(t) dt (33)$$

$$[X_e] = \log(T_{wb}^{-1} * T_{di}) \quad (34) \text{ (προκύπτει } 4 \times 4 \text{ πίνακας άρα για να βρούμε το διάνυσμα } X_e \text{ αξιοποιούμε τις (18), (19)).}$$

Για τις τιμές των παραμέτρων (κέρδη) K_p και K_I υπάρχουν διάφορες τεχνικές (πχ στο μάθημα των ΣΑΕ διδαχθήκαμε την τεχνική *Ziegler-Nichols κ.α*), ωστόσο εμείς κάνουμε την παρακάτω απλή θεώρηση:

$$K_p = 4 \quad (35)$$

$$K_d = 0.01 * K_p = 0.04 \quad (36)$$

Ως απόρροια των παραπάνω, κάθε χρονική στιγμή, το *twist* του *end-effector* θα δίνεται βάσει της εξίσωσης:

$$V_b(t) = [Ad_{T_{bw}}] * V_d(t) + V_{PI}(t) = [Ad_{T_{bw}}] * V_d(t) + 4 * X_e(t) + 0.04 * \int_0^t X_e(t) dt \quad (37)$$

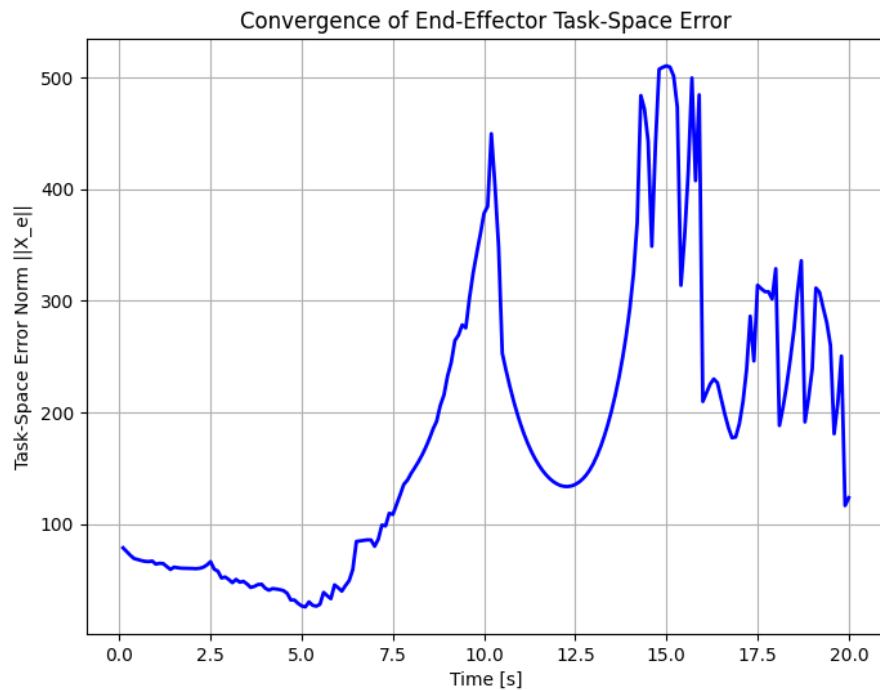
Τέλος, για τον *adjoint* μετασχηματισμό του πίνακα $T = (R, p)$ (*rotation matrix και translation vector αντιστοίχως*) θα είναι:

$$[Ad_T] = \begin{bmatrix} R & 0 \\ \hat{p} * R & R \end{bmatrix} \in R^{6 \times 6} \quad (38) \text{ (}\hat{p}\text{) είναι ο skew - syymetric matrix του } 3 \times 1 \text{ διανύσματος θέσης } p = (p_x, p_y, p_z)^T$$

Ερώτημα 8: Κώδικας /Αποτελέσματα

(Τρέχουμε το αρχείο *simulator.py*)

Το 2^ο *figure* (το πρώτο παρουσιάζει το σχετικό animation) το οποίο παρουσιάζει το πώς συγκλίνει το σφάλμα ανάμεσα στην επιθυμητή και στην πραγματική θέση του *end-effector* έχει ως εξής:



Να σημειωθεί ότι ο κώδικας δημιουργήθηκε από το *ChatGPT* με συνεχές *prompting* του γράφοντος το παρόν με τη συνδρομή παράλληλα και κώδικα του διδάσκοντα.

Παρατηρήσεις για τον κώδικα:

- 1) Δεν κατάφερα να κάνω τον κύλινδρο να μετακινείται μαζί με τον *end-effector*
- 2) Για να σταματήσει το *animation* ενδεχομένως να απαιτείται χρόνος ~ 1 λεπτού
- 3) Στο 1 figure όπου παρουσιάζεται το *animation*, απεικονίζεται παράλληλα και τα αρχικό configuration του ρομποτικού βραχίονα
- 4) Η κίνηση του ρομποτικού βραχίονα δεν είναι και ιδιαίτερα ομαλή, κάτι το οποίο μπορεί να οφείλεται στους παρακάτω παράγοντες:
- 5) Ο κύλινδρος έχει λανθασμένο προσανατολισμό

- a) Κάποιο ελάσσον/μείζον λάθος στον κώδικα (πχ αριθμητικό)
 - b) Λανθασμένη εφαρμογή της θεωρίας
 - c) Μέθοδος ολοκλήρωσης
 - d) Παράμετροι του ΡΙ ελεγκτή.
 - e) Στον αλγόριθμο ολοκλήρωσης
 - f) Λανθασμένοι πίνακες έναρξης/προορισμού
- 6) Στο αρχείο requirements.txt καταγράφονται όλες οι εξωτερικές βιβλιοθήκες οι οποίες πρέπει να γίνουν install
- 7) Προσπάθησα ανεπιτυχώς πολυάριθμες φορές να διορθώσω τον κώδικα ρωτώντας συμφοιτητές μου/τον καθηγητή αλλά προέκυπταν εν συνεχεία άλλα εμπόδια..